FIAP ON

BUILDING STATISTICAL MODELS

METODOLOGIA DE VALIDAÇÃO CRUZADA E PROCESSO DE SELEÇÃO DE VARIÁVIES

2

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Visualização de viés e variância 9
Figura 2 – Diferenças resultados treino e validação.....11
Figura 3 – Método da validação cruzada 12
Figura 4 – Dados análise PSA 17
Figura 5 – Correlações análise PSA 18
Figura 6 – Resultados exemplo validação cruzada: índices.....20
Figura 7 – Coeficientes estimados da regressão *Ridge* 21
Figura 8 – Coeficientes estimados da regressão *LASSO* 22
Figura 9 – Resultados das avaliações dos modelos 25

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Ferramentas Python 15

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Importar módulos iniciais e dados.....16
Código-fonte 2 – Dados dos exemplos 16
Código-fonte 3 – Preparação dos dados.....18
Código-fonte 4 – Exemplo validação cruzada: índices.....19
Código-fonte 5 – Aplicar regressão *Ridge*.....20
Código-fonte 6 – Coeficientes regressão *Ridge*.....20
Código-fonte 7 – Aplicar regressão *LASSO*.....21
Código-fonte 8 – Coeficientes regressão *LASSO* 22
Código-fonte 9 – Validação cruzada na regressão *Ridge*.....23
Código-fonte 10 – Validação cruzada na regressão *LASSO*.....23
Código-fonte 11 – *Tunning* regressão Ridge.....23
Código-fonte 12 – *Tunning* regressão LASSO 24
Código-fonte 13 – Obtenção dos resultados finais dos modelos.....24

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 METODOLOGIA DE VALIDAÇÃO CRUZADA E PROCESSO DE SELEÇÃO DE VARIÁVIES 6
1.1 Introdução 6
1.2 Principais elementos de aprendizagem supervisionada.....8
1.3 Conceitos de validação cruzada.....10
1.4 Conceitos de regressão penalizada 13
2 APLICAÇÕES PRÁTICAS EM VALIDAÇÃO CRUZADA E REGRESSÃO PENALIZADA 15
2.1 Principais funções no *Python* 15
2.2 Dados e exemplos.....16
2.3 Análise exploratória e preparação dos dados 17
2.4 Validação cruzada no *Python*.....18
2.5 Regressão penalizada no *Python*.....20
2.6 Regressão penalizada com validação cruzada no *Python* 22
ATIVIDADE 26
REFERÊNCIAS.....27

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 METODOLOGIA DE VALIDAÇÃO CRUZADA E PROCESSO DE SELEÇÃO DE VARIÁVIES

1.1 Introdução

Um analista de dados, após descrever os dados e verificar a existência de correlação entre variáveis, buscará entender padrões preditivos. Ou seja, a partir do momento que um pesquisador verifica uma associação entre duas ou mais variáveis, ele poderá testar a possibilidade de predizer o valor de uma por meio da outra.

No contexto do *Machine Learning* (ML), a forma mais comum de trabalhar com tarefas preditivas é a utilização de técnicas supervisionadas. De acordo com Hastie Tibshirani e Friedman (2009), o objetivo da aprendizagem supervisionada é utilizar *inputs* (variáveis que podem ter influência em alguma determinada saída) para realizar previsões em algum determinado *output* (a saída ou resultado esperado).

Apesar da importância estabelecida dos modelos de ML nas últimas décadas, realizar o treinamento e a validação dos algoritmos não é uma tarefa simples. Uma etapa essencial é dividir os dados em amostras de treino e teste. Contudo, em muitos dos casos, essa divisão não é suficiente para garantir a maior generalização do modelo.

Conforme Hastie Tibshirani e Friedman (2009), a generalização da capacidade preditiva de um algoritmo de ML precisa ser avaliada em dados de teste completamente separados das amostras de treino e validação. Além disso, nesta etapa podemos garantir a melhor escolha do modelo, bem como mensurar a qualidade dos resultados.

Essa qualidade é direcionada pelo conceito de estabilidade do algoritmo, isto é, a capacidade de o modelo conseguir se adaptar a dados ainda não observados durante o treinamento. Deste modo, busca-se averiguar se o modelo apresenta baixos valores de **viés** e **variância**.

Conforme Zou e Hastie (2005), um modelo apresentará menor **viés** quando os valores previstos estão muito próximos, em média, dos verdadeiros valores observados. Em média, os valores podem ser próximos, existindo, portanto, o risco de os valores previstos apresentarem uma variabilidade significativamente alta,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

conforme novos dados são incorporados no banco de dados. Quando isso ocorre, afirmamos que o modelo possui alta **variância**.

Além do problema de manter a qualidade do modelo, um analista de dados precisa interpretar os resultados do ponto de vista dos negócios. Isso posto, um outro objetivo precisa ser otimizado, no caso, a **complexidade** do modelo. Quanto mais complexo um algoritmo de ML, mais difícil se torna a interpretação.

Este ponto é discutido por James, Witten, Hastie e Tibshirani (2013), em que, dado a vasta quantidade de colunas que podem ser exploradas num banco de dados, existe a possibilidade de inclusão de variáveis pouco relevantes num modelo.

Dessa forma, o uso exagerado de variáveis irrelevantes num algoritmo elevará de forma desnecessária a complexidade, o que dificulta a validade para o negócio. Uma vez que explicar esse modelo para um cliente pouco experiente se torna demasiadamente difícil, além de complicar a adoção e confiabilidade nas predições propostas.

Visando trazer possíveis soluções para o problema da complexidade, Hastie, Tibshirani e Friedman (2009) apresentam diversos métodos de seleção de variáveis, os quais tanto visam minimizar a complexidade quanto o viés e variância (complexidade) dos modelos de *Machine Learning*.

Diante dos problemas abordados (**viés** e **complexidade**), o objetivo deste capítulo é apresentar metodologias com o fim de contornar desafios e prover métodos que possam auxiliar analistas de dados a melhor realizarem treinamentos de seus algoritmos.

No caso, devido à grande disponibilidade de técnicas disponíveis para solucionar esses problemas, focaremos em duas metodologias bastante adotadas por cientistas de dados experientes. Para o problema de minimizar os problemas de **viés,** apresentaremos o método da validação cruzada. Em relação à **complexidade**, estudaremos as técnicas de regularização *Lasso* e *Ridge*.

Nos próximos tópicos, serão apresentados os principais conceitos relacionados aos elementos de aprendizagem supervisionada, validação cruzada e penalização de modelos de ML. Também serão apresentados alguns exemplos práticos utilizando o *Python*, bem como as principais funções utilizadas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.2 Principais elementos de aprendizagem supervisionada

No contexto do *Machine Learning* (ML), a forma mais comum de trabalhar com tarefas preditivas é a utilização de técnicas supervisionadas. De acordo com Hastie, Tibshirani e Friedman (2009), o objetivo da **aprendizagem supervisionada** é utilizar *inputs* (variáveis que podem ter influência em alguma determinada saída) para realizar previsões em algum determinado *output* (a saída ou resultado esperado).

Do ponto de visto analítico, um dos principais objetivos de aprendizagem supervisionada é otimizar a função objetivo:

$$\mathrm{obj}(\theta) = \mathrm{L}(\theta) + \Omega(\theta). \tag{1}$$

Em que, $\mathrm{L}(\theta)$ é a função de aprendizagem (*loss function*), esta mensura a qualidade de ajuste (**viés**) do modelo nos dados de treino. No caso, $\Omega(\theta)$ é a função de penalização (regularização), está mede a complexidade (**variância**) do modelo.

Existem vários tipos de funções de aprendizagem, porém as mais conhecidas são apresentadas nas expressões 2 e 3, sendo, quadrática e logística, respectivamente:

$$\mathrm{L}(\theta) = \sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \tag{2}$$

$$\mathrm{L}(\theta) = y_i \ln\left(1 + e^{-\hat{y}_i}\right) + (1 - y_i)\ln\left(1 + e^{-\hat{y}_i}\right). \tag{3}$$

Em que, y_i é o valor observado e é o valor previsto $\hat{y}_i$. Quando levantamos as métricas de regularização, podemos citar as funções L1 e L2, representadas nas expressões 4 e 5, respectivamente:

$$\mathrm{L1} = \Omega(w) = \lambda\|w\|_1, \tag{4}$$

$$\mathrm{L2} = \Omega(w) = \lambda\|w\|^2. \tag{5}$$

Quando o analista busca otimizar o objetivo da função de aprendizagem, o foco é melhorar a capacidade de previsão do modelo. Quanto a otimizar a função de penalização, busca-se melhorar a simplicidade do modelo. Modelos mais simples,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

além de serem fáceis de explicar para clientes menos técnicos, tendem a demonstrar previsões mais estáveis.

Estes objetivos em modelos supervisionados visam equilibrar o viés e a variância dos algoritmos, na Figura “Visualização de viés e variância”, explicamos de forma simples como estes conceitos são observados em treinamentos de algoritmos.

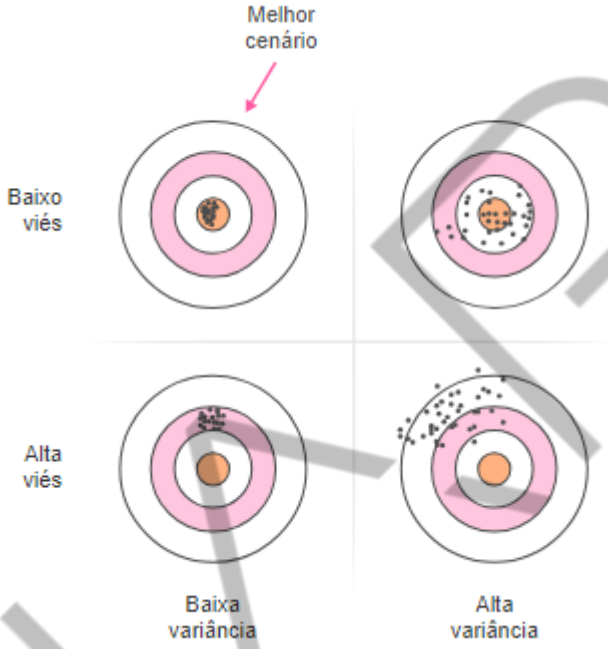

Figura 1 – Visualização de viés e variância
Fonte: Baseado em Zou e Hastie (2005)

A função objetivo 1 buscará encontrar o melhor caso, no qual seria baixo viés e variância, conforme apontado na figura. Contudo, quando o modelo não é bem ajustado, teremos cenários similares aos demais exemplos demonstrados.

Caso o analista não seguir o rigor metodológico, há o risco de uso demasiado de variáveis, diminuindo o viés, porém aumentando a variância. Por outro lado, poucas variáveis poderão trazer menor variância para o modelo, mas o viés inviabilizará o uso em situações diárias nos negócios.

As técnicas de validação cruzada e penalização serão fundamentais para auxiliar o analista a melhorar seus resultados e encontrar os melhores cenários preditivos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3 Conceitos de validação cruzada

Uma das estratégias mais utilizadas para ajustar modelos supervisionados de *Machine Learning* é dividir os dados em amostras de treino, validação e teste. A amostra de treino é utilizada para ajustar os modelos; a de validação é usada para seleção do melhor algoritmo e a amostra de teste é aproveitada para tomar a decisão final. Sendo que o uso da amostra de teste deve ser realizado somente na etapa final da aplicação do algoritmo.

Conforme James, Witten, Hastie e Tibshirani (2013), não há uma regra sobre como dividir os dados. Porém, costuma-se manter as proporções como 50% para treino, 25% para validação e outros 25% para teste. Nesta etapa, funções como as apresentadas em 2 (quadrática) e 3 (logística) são utilizadas para verificar o viés do modelo.

Apesar de esta abordagem ser conceitualmente bastante simples, além de fácil implementação, podemos citar alguns pontos limitantes. O primeiro seria a existência de dados insuficientes para realizar a divisão. Por exemplo, caso existam somente 100 linhas de dados, 50 seriam para treino e as outras 25 para validação e teste, respectivamente.

Um segundo ponto seria a grande variabilidade nos índices de validação devido às observações que foram incluídas nas amostras de treino e validação. Por exemplo, num primeiro momento, acurácia pode ser de 80%; porém, numa segunda validação, esse valor pode cair para 30%. Dessa forma, torna-se complicado confiar num algoritmo que apresenta muita variação.

Um terceiro problema ocorre devido ao tamanho da amostra em cada divisão. Modelos estatísticos tendem a melhorar quando são treinados em amostras maiores. Contudo, aumentar a amostra de treino diminuirá as partes de validação e teste. Diante disto, existe o risco de superestimar ou subestimar o erro do modelo nessas amostras.

Na Figura “Diferenças resultados treino e validação”, demonstramos como as métricas de avaliação podem apresentar muita diferença nas amostras de treino e validação. Conforme aumenta o grau de complexidade do algoritmo, mais variabilidade é observada nos resultados, tornando difícil a implementação do modelo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

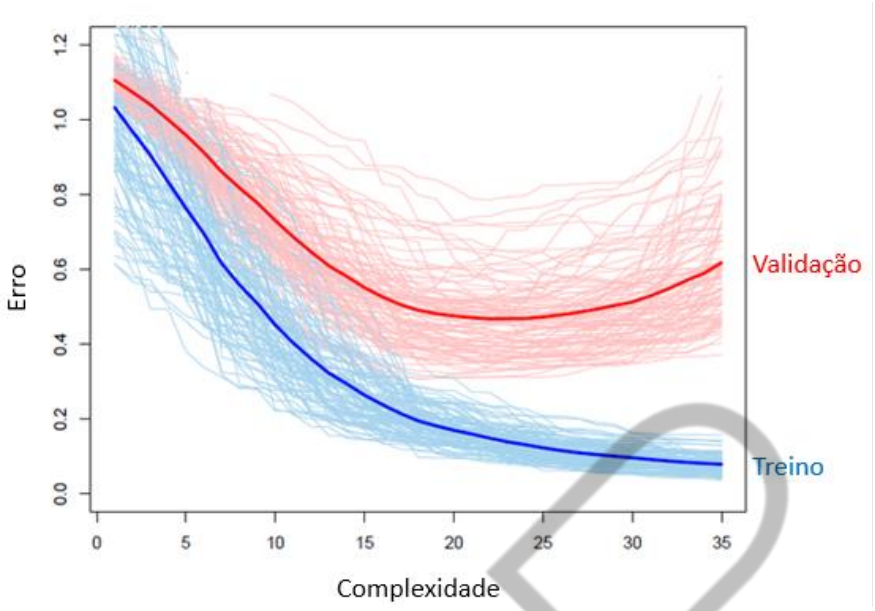

Figura 2 – Diferenças resultados treino e validação
Fonte: Baseado em Hastie, Tibshirani e Friedman (2009)

Visando superar esse problema, foram propostos métodos baseados em reamostragem. Nesta metodologia de reamostragem, busca-se obter amostras dos dados repetidamente e reajustar novamente o modelo. Assim, pode-se obter maior capacidade de generalização do algoritmo supervisionado, uma vez que o modelo é retreinado várias vezes.

No século passado, esse tipo de abordagem era algo impensável devido aos altos custos computacionais. Contudo, por meio do crescente poder de computação, o uso desse método se tornou corriqueiro nas atividades de analistas e cientistas de dados. Dentro as variadas metodologias, a técnica de **validação cruzada** (*cross-validation*) tornou-se uma das mais utilizadas.

Conforme Gençay, Selçuk e Whitcher (2001) essa técnica era de conhecimento da comunidade científica desde os anos 1930. Porém o refinamento e a popularização ocorreram nos anos 1970, nos trabalhos de Geisser (1975) e Stone (1974), sendo neste último derivado o termo "*cross-validating methodology*".

Podemos citar dois tipos de validação cruzada: *K-fold* e *Stratified K-fold*. Ambos os métodos são similares, sendo que o segundo busca corrigir problemas de desbalanceamento. Por exemplo, em problemas de classificação podem existir mais exemplos negativos que positivos ou, em problemas de regressão, termos linhas contendo muitos valores altos em contrapartida de poucos valores baixos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

À vista disto, na validação cruzada, busca-se dividir os dados em *k* subgrupos de tamanhos aproximadamente iguais. O primeiro bloco é tratado como amostra de validação e o modelo é ajustado nos demais *k-1* blocos. Alguma métrica de validação (Acurácia e Erro médio quadrático são exemplos) é obtida no subgrupo de validação.

Este processo precisa ser repetido *k* vezes, sendo que a cada etapa, uma amostra de validação diferente é utilizada para obter as métricas de avaliação dos modelos. No último passo, deve-se obter os resultados das etapas anteriores e gerar um valor médio que representará a performance do algoritmo supervisionado. Na Figura “Método da validação cruzada”, podemos visualizar as etapas de forma detalhada.

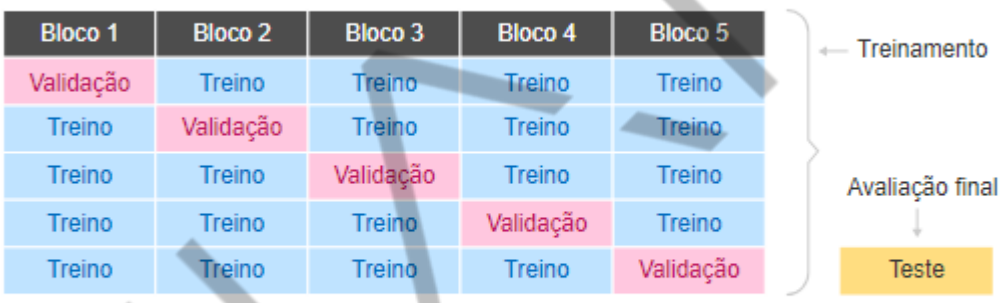

Figura 3 – Método da validação cruzada
Fonte: Baseado em James, Witten, Hastie e Tibshirani (2013)

Após entendermos o conceito, podemos formalizar o método conforme as seguintes etapas:

1) Dividir os dados aleatoriamente em *k* subgrupos.

2) Para cada subgrupo k = 1, 2, …, K:

- Encontrar as variáveis *X* que possuem maior correlação com a variável *y*.
- Treinar o modelo utilizando as variáveis que possuem correlação significativa com *y*.
- Utilizar o modelo treinado para gerar as predições.
- Obter as métricas de validação.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3) Realizar a avaliação final na amostra de teste que foi separada no início das análises.

1.4 Conceitos de regressão penalizada

Quando avaliamos o equilíbrio de viés e complexidade (evitar o *overfitting*), além das etapas da validação cruzada, conseguimos observar, no mínimo, dois problemas. O primeiro seria o tamanho necessário da amostra para realizar o procedimento. Uma vez que, caso o subgrupo *k* possuir menor número de linhas (n) que variáveis (p), isto é, n < p, torna-se impossível treinar muitos dos algoritmos supervisionados.

O segundo é o processo de encontrar as variáveis *X* que possuem maior correlação com a variável *y* e, depois, treinar o modelo utilizando as variáveis que possuem correlação significativa. Conforme Hastie, Tibshirani e Friedman (2009) e James, Witten, Hastie e Tibshirani (2013), a correlação entre as variáveis *X* e *y* podem variar significativamente para cada subgrupo *k* da validação cruzada. Desta forma, o modelo é treinado por variáveis diferentes em cada etapa, o que dificulta a seleção final do algoritmo.

Diante desse cenário, deve-se pensar num método que auxilie no processo de seleção de variáveis, mesmo na existência dos problemas citados. Por conseguinte, os métodos de encolhimento (*shrinkage*) se tornam importantes. Nessa abordagem, em todas as etapas de reamostragem (como na validação cruzada), todas as variáveis são utilizadas, não há eliminação. Porém as variáveis X que possuem baixa correlação com *y* terão pesos atribuídos próximos de zero.

Nesse cenário, as técnicas mais conhecidas e utilizadas são as de penalização (regularização). Nesses processos, as variáveis recebem o mesmo peso inicialmente e, a cada etapa, mais peso (em pequenas “quantidades”) é atribuído para as variáveis que melhor contribuem para minimizar o erro. Pesos grandes somente serão permitidos se diminuírem de forma significativa os valores da função de aprendizagem (funções similares as expressões 2 e 3).

Os métodos de penalização mais conhecidos são os denominados de *Ridge* e *LASSO*. O primeiro foi proposto em Hoerl e Kennard (1988), os autores propõem uma

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

estratégia de penalização baseada em função do tipo L2, conforme foi apresentada expressão 5.

Deste modo, o método *Ridge* busca minimizar o erro, porém elevando ao quadrado os pesos das variáveis, fazendo com que a função final de custo se torne maior, caso pesos muito grandes sejam associados às variáveis. Assim, a complexidade do algoritmo é controlada, minimizando os riscos de superajustes (*overfitting*). O método *Ridge* é demonstrado na expressão 6:

$$\hat{\beta}^{ridge} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^{N} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \alpha \sum_{j=1}^{p} \beta_j^2 \right\}. \tag{6}$$

Em que, β são os coeficientes e α controla o grau de penalização, quanto maior o valor de α maior a penalidade que o algoritmo atribuirá aos pesos.

Um outro método bastante usado é o denominado *LASSO* (***L**east **A**bsolute **S**hrinkage and **S**election **O**perator*), proposto por Tibshirani (1996), nesta abordagem, busca-se realizar a penalização por meio de uma função do tipo L1, conforme foi apresentada na expressão 4.

Segundo o autor, além de realizar o processo de regularização, o método realiza uma seleção automática das variáveis mais relevantes para o modelo. Em relação à performance dos algoritmos, não há diferença significativa entre o LASSO e o Ridge. Porém a seleção automática das variáveis torna a ferramenta mais prática para os analistas de dados.

Na expressão 7, apresentamos o método LASSO:

$$\hat{\beta}^{lasso} = \underset{\beta}{\operatorname{argmin}} \left\{ \frac{1}{2} \sum_{i=1}^{N} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \alpha \sum_{j=1}^{p} |\beta_j| \right\}. \tag{7}$$

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 APLICAÇÕES PRÁTICAS EM VALIDAÇÃO CRUZADA E REGRESSÃO PENALIZADA

Nos próximos tópicos, serão trabalhados e detalhados os conceitos citados, vamos aplicar algumas das ferramentas como exemplo prático no *Python*; e conhecer as principais funções, os dados e aplicar os métodos de validação cruzada, regressão *Ridge* e *LASSO* em modelo de *Machine Learning*.

2.1 Principais funções no *Python*

Como ferramentas necessárias, os módulos *numpy* e *pandas* serão as principais bibliotecas do *Python* novamente. As bibliotecas *matplotlib* e *seaborn* serão novamente utilizadas para análise gráfica.

Para realizar as regressões penalizadas e validação, utilizaremos a ferramenta *scikit-learn*. Esta é uma das ferramentas mais utilizadas em *Machine Learning*. No Quadro "Ferramentas *Python*", apresentamos um resumo das principais funções que serão utilizadas nos exemplos.

Podemos destacar as funções Ridge e Lasso que aplicarão os modelos apresentados e as funções KFold e cross_val_score servirão para realizar a validação cruzada.

Módulos	Funções	Aplicações
Matplotlib	Várias funções serão utilizadas	Análise gráfica
Seaborn	Várias funções serão utilizadas	Análise gráfica
Numpy	Várias funções serão utilizadas	Análise numérica
Pandas	Várias funções serão utilizadas	Leitura e tratamento de dados
Scikit-Learn	train_test_split	Dividir os dados amostra treino e teste
	cross_val_score	Realizar validação cruzada
	KFold	Obter os índices para realizar a validação cruzada
	StratifiedKFold	Obter os índices para realizar a validação cruzada estratificada
	Ridge	Aplicar regressão Ridge
	Lasso	Aplicar regressão LASSO
	RidgeCV	*Tunning* da regressão Ridge com validação cruzada
	LassoCV	*Tunning* da regressão Lasso com validação cruzada
	mean_squared_error	Obter a métrica do erro médio quadrático

Quadro 1 – Ferramentas *Python*
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Dados e exemplos

Novamente utilizaremos o *Colab*: digite, no navegador, https://colab.research.google.com/, para criar um novo notebook e iniciar as análises.

Nos exemplos a seguir, utilizaremos os dados “psa_cancer_data.csv”. O objetivo é aplicar o processo de validação cruzada, combinando as práticas das regressões Ridge e LASSO.

Os códigos para importar os módulos e carregar os dados podem ser vistos no Código-fonte “Importar módulos e dados”. Com os códigos apresentados neste texto, também é possível analisar os códigos de forma mais detalhada no arquivo “cap_validacao_cruzada.ipynb” disponibilizado.

```
# gerais
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# validacao cruzada
from sklearn.model_selection import KFold, StratifiedKFold

# regressao penalizada
from sklearn.linear_model import Ridge, Lasso, RidgeCV, LassoCV
from sklearn.model_selection import train_test_split, cross_val_score

# avaliacao
from sklearn.metrics import mean_squared_error

# carregar dados
dados = pd.read_csv('psa_cancer_data.csv')
```

Código-fonte 1 – Importar módulos iniciais e dados
Fonte: Elaborado pelo autor (2022)

No Código-fonte “Dados dos exemplos”, podemos observar como obter as cinco primeiras linhas do *dataset* que será utilizado.

```
# dados gastos
dados.head()
```

Código-fonte 2 – Dados dos exemplos
Fonte: Elaborado pelo autor (2022)

Na Figura “Dados análise PSA”, podemos observar as cinco primeiras linhas do *dataset*. Temos 9 variáveis em que “lpsa” será a variável dependente, e as demais serão candidatas a estarem nos modelos *Ridge* e *LASSO*. Esse banco de dados é

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

proveniente de Hastie, Tibshirani e Friedman (2009), estes dados replicam um estudo sobre análise de câncer de próstata.

- lcavol: logaritmo do volume do câncer.
- lweight: logaritmo do peso da próstata.
- age: idade do paciente.
- lbph: logaritmo da hiperplasia prostática benigna.
- svi: invasão da vesícula seminal.
- lcp: logaritmo da penetração capsular.
- gleason: escala de Gleason.
- pgg45: percentil da escala Gleason.
- lpsa: logaritmo do antígeno prostático específico (*target*).

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45	lpsa
0	-0.579818	2.769459	50	-1.386294	0	-1.386294	6	0	-0.430783
1	-0.994252	3.319626	58	-1.386294	0	-1.386294	6	0	-0.162519
2	-0.510826	2.691243	74	-1.386294	0	-1.386294	7	20	-0.162519
3	-1.203973	3.282789	58	-1.386294	0	-1.386294	6	0	-0.162519
4	0.751416	3.432373	62	-1.386294	0	-1.386294	6	0	0.371564

Figura 4 – Dados análise PSA
Fonte: Elaborado pelo autor (2022)

2.3 Análise exploratória e preparação dos dados

Antes de iniciarmos os exemplos relacionados à validação cruzada e regressão Ridge/LASSO, torna-se importante realizar uma breve análise exploratória. Na Figura “Correlações análise PSA”, apresentamos a matriz de correlação das variáveis e destacamos as correlações entre as variáveis e lpsa.

Percebemos que todas as variáveis possuem correlação positiva com lpsa, sendo que lcavol, svi e lcp são as três que apresentaram os maiores valores, sendo as principais candidatas a melhor explicarem a relação com a variável dependente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45	lpsa
lcavol	1.00	0.28	0.22	0.03	0.54	0.68	0.43	0.43	0.73
lweight	0.28	1.00	0.35	0.44	0.16	0.16	0.06	0.11	0.43
age	0.22	0.35	1.00	0.35	0.12	0.13	0.27	0.28	0.17
lbph	0.03	0.44	0.35	1.00	-0.09	-0.01	0.08	0.08	0.18
svi	0.54	0.16	0.12	-0.09	1.00	0.67	0.32	0.46	0.57
lcp	0.68	0.16	0.13	-0.01	0.67	1.00	0.51	0.63	0.55
gleason	0.43	0.06	0.27	0.08	0.32	0.51	1.00	0.75	0.37
pgg45	0.43	0.11	0.28	0.08	0.46	0.63	0.75	1.00	0.42
lpsa	0.73	0.43	0.17	0.18	0.57	0.55	0.37	0.42	1.00

Figura 5 – Correlações análise PSA
Fonte: Elaborado pelo autor (2022)

Outra etapa importante antes da aplicação dos modelos é a preparação dos dados. No caso, vamos separar as variáveis X e a variável y. Além disso, precisamos separar os dados para treino, validação e teste; sendo que este último será utilizado somente na última etapa das análises.

No Código-fonte “Preparação dos dados” temos o processo de seleção das variáveis e divisão dos dados. Para o exemplo, separamos 20 linhas para a avaliação final dos modelos na amostra de teste.

```
# selecao das variaveis
cols_x = ['lcavol', 'lweight', 'age', 'lbph', 'svi', 'lcp', 'gleason',
 'pgg45']

col_y = ['lpsa']

X = dados[cols_x]
y = dados[col_y]

# divisao dos dados
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 20,
                                                    random_state = 12345)
```

Código-fonte 3 – Preparação dos dados
Fonte: Elaborado pelo autor (2022)

2.4 Validação cruzada no *Python*

Neste exemplo, apresentaremos como a validação cruzada é realizada no *Python*. Por enquanto, somente o processo será demonstrado, ou seja, nenhum

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

modelo será treinado neste momento. O foco é entender como os blocos de validação cruzada são criados para aplicação em análises futuras.

Para realizar esta demonstração, utilizaremos a função *KFold*, em que criaremos 5 blocos para geração das validações cruzadas. No Código-fonte “Exemplo validação cruzada: índices” instanciamos a função *KFold*, depois aplicamos um *loop* para obter os índices que serão filtrados no *dataframe*.

```
# funcao
crossKfold = KFold(n_splits=5, shuffle=True, random_state = 12345)

# indices dos dados
k = 0
for train, val in crossKfold.split(X_train, y_train):

  k += 1
  print(f'----Kfold: {k}')

  # indices do treino
  print('--Treino')
  print(train)

  # indices da validacao
  print('--Validacao')
  print(val)
```

Código-fonte 4 – Exemplo validação cruzada: índices
Fonte: Elaborado pelo autor (2022)

Conforme foi aplicado no código, uma função *split* foi usada de forma que, a cada iteração, sejam geradas as amostras de treino e validação no bloco da *cross-validation*. Um detalhe importante a ser citado é que utilizamos os objetos X_train e y_train; é por meio desses dados que criamos os subgrupos para realizar a validação cruzada, uma vez que X_test e y_test utilizaremos somente no final do estudo.

Na Figura “Resultados exemplo validação cruzada: índices” apresentamos os resultados do código aplicado. Para cada iteração, conseguimos observar os índices a serem filtrados no *dataset* de treinamento.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
----Kfold: 1
--Treino
[ 0  1  3  5  7  8  9 10 11 12 14 15 16 17 18 19 21 22 23 24 26 27 29 31
 32 33 34 35 36 38 39 40 41 42 43 44 45 46 47 49 51 52 53 55 56 57 58 59
 61 63 64 65 66 67 68 69 71 72 74 75 76]
--Validacao
[ 2  4  6 13 20 25 28 30 37 48 50 54 60 62 70 73]
----Kfold: 2
--Treino
[ 0  1  2  3  4  5  6  7  8 10 11 12 13 14 15 16 17 19 20 23 24 25 27 28
 29 30 31 33 34 36 37 38 39 40 41 42 43 44 46 47 48 49 50 54 55 56 58 59
 60 61 62 63 67 68 70 71 72 73 74 75 76]
--Validacao
[ 9 18 21 22 26 32 35 45 51 52 53 57 64 65 66 69]
```

Figura 6 – Resultados exemplo validação cruzada: índices
Fonte: Elaborado pelo autor (2022)

2.5 Regressão penalizada no *Python*

Após entender o processo de validação cruzada no *Python*, podemos conhecer como realizar as regressões *Ridge* e LASSO. Em ambas as funções, precisamos adicionar um parâmetro denominado alpha, conforme pode ser visto nas expressões 6 e 7.

No Código-fonte “Aplicar regressão *Ridge*”, temos o processo para treinar um modelo de ML utilizando a função *Ridge*. Vale ressaltar que, nesta etapa, treinamos o modelo por meio do uso de todos os dados, ainda sem validação cruzada. Também aplicamos a função assumindo alpha = 1.

```
# regressao ridge
alpha_ = 1 # controla o nivel de regularizacao
regressao_ridge = Ridge(alpha = alpha_)
regressao_ridge.fit(X_train, y_train)
```

Código-fonte 5 – Aplicar regressão *Ridge*
Fonte: Elaborado pelo autor (2022)

Após treinar o modelo, podemos obter os coeficientes utilizando o Código-fonte “Coeficientes regressão *Ridge*”. Neste processo, conseguimos obter os respectivos coeficientes das colunas usadas e o intercepto do algoritmo de regressão.

```
# coeficientes ridge
betas = np.append(regressao_ridge.intercept_, regressao_ridge.coef_)
colunas = ['intercepto']+X.columns.to_list()
coeficientes_ridge = pd.DataFrame(betas, colunas, columns=['Coeficient
es'])
coeficientes_ridge
```

Código-fonte 6 – Coeficientes regressão *Ridge*
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na Figura “Coeficientes estimados da regressão *Ridge*”, temos os resultados dos coeficientes da regressão *Ridge*. Conseguimos observar que “lcavol”, “lweight” e “svi” receberam os maiores pesos, em conformidade com a análise de correlação realizada anteriormente, na etapa de análise exploratória.

	Coeficientes
intercepto	0.823461
lcavol	0.658057
lweight	0.435113
age	-0.020541
lbph	0.080840
svi	0.740854
lcp	-0.128140
gleason	0.025019
pgg45	0.004476

Figura 7 – Coeficientes estimados da regressão *Ridge*
Fonte: Elaborado pelo autor (2022)

No Código-fonte “Aplicar regressão *LASSO*”, temos o processo para treinar um modelo de ML utilizando a função *Lasso*. Ressaltamos que o modelo será treinado com todos os dados somente para conhecer a função. Também aplicamos a função assumindo alpha = 1.

```
# regressao lasso
alpha_ = 1 # controla o nivel de regularizacao
regressao_lasso = Lasso(alpha = alpha_)
regressao_lasso.fit(X_train, y_train)
```

Código-fonte 7 – Aplicar regressão *LASSO*
Fonte: Elaborado pelo autor (2022)

Os coeficientes para a regressão LASSO podem ser obtidos por meio do Código-fonte “Coeficientes regressão *LASSO*”. Os resultados podem ser visualizados na Figura “Coeficientes estimados da regressão *LASSO*”. Um detalhe importante é que o valor de alpha = 1 no modelo LASSO é bastante restritivo, fazendo com que somente a variável “pgg45” teve valor acima de zero.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# coeficientes lasso
alpha_ = 1 # controla o nivel de regularizacao
regressao_lasso = Lasso(alpha = alpha_)
regressao_lasso.fit(X_train, y_train)
```

Código-fonte 8 – Coeficientes regressão *LASSO*
Fonte: Elaborado pelo autor (2022)

	Coeficientes
intercepto	2.050267
lcavol	0.000000
lweight	0.000000
age	0.000000
lbph	0.000000
svi	0.000000
lcp	0.000000
gleason	0.000000
pgg45	0.018824

Figura 8 – Coeficientes estimados da regressão *LASSO*
Fonte: Elaborado pelo autor (2022)

2.6 Regressão penalizada com validação cruzada no *Python*

Nos tópicos anteriores, foram apresentados os processos para realizar a validação cruzada e como estimar regressões Ridge e LASSO. Agora, vamos demonstrar a combinação dos métodos para otimizar a capacidade de aprendizagem de algoritmos supervisionados.

Nos Códigos-fonte “Validação cruzada na regressão *Ridge*” e “Validação cruzada na regressão *LASSO*” utilizamos a função *cross_val_score* para realizar a validação cruzada usando os modelos de regressão Ridge e LASSO.

Detalhe importante: no parâmetro “cv” foi possível utilizar o objeto “crossKfold” criado no processo de geração das amostras para validação cruzada. Na última linha, conseguimos obter a raiz do erro médio quadrado de todos os subgrupos gerados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# funcao cross_val_score automatiza todo o processo de cross-
validation
alpha_ = 1 # controla o nivel de regularizacao
regressao_ridge = Ridge(alpha = alpha_)
score_ridge = cross_val_score(regressao_ridge, X_train, y_train,
                              cv = crossKfold,
                              scoring = 'neg_mean_squared_error')
regressao_ridge.fit(X_train, y_train)
np.sqrt(np.mean(-1 * score_ridge))
```

Código-fonte 9 – Validação cruzada na regressão *Ridge*
Fonte: Elaborado pelo autor (2022)

```
# funcao cross_val_score automatiza todo o processo de cross-
validation
alpha_ = 1 # controla o nivel de regularizacao
regressao_lasso = Lasso(alpha = alpha_)
score_lasso = cross_val_score(regressao_lasso, X_train, y_train,
                              cv = crossKfold,
                              scoring = 'neg_mean_squared_error')
regressao_lasso.fit(X_train, y_train)
np.sqrt(np.mean(-1 * score_lasso))
```

Código-fonte 10 – Validação cruzada na regressão *LASSO*
Fonte: Elaborado pelo autor (2022)

Por meio da avaliação dos códigos, percebemos que utilizamos o parâmetro alpha = 1. Um processo importante é encontrar uma forma de definir esse valor por meio de otimização. Nos Códigos-fonte "*Tunning* regressão Ridge" e "*Tunning* regressão *LASSO*", apresentamos como realizar o processo de otimização de alpha nos modelos.

Deste modo, as funções RidgeCV e LassoCV nos auxiliam nesta tarefa, uma vez que alguns valores de alpha podem ser testados durante um processo de validação cruzada. Assim, podemos encontrar os valores ótimos do parâmetro alpha e utilizá-lo para treinar os modelos.

```
# otimizar alpha
regressao_ridge_otimizada = RidgeCV(alphas = [0.001, 0.01, 0.1, 0.5, 0
.7, 1, 7, 10], cv=crossKfold)
score_ridge_otimizado = cross_val_score(regressao_ridge_otimizada, X_t
rain, y_train,
                                        cv = crossKfold,
                                        scoring = "neg_mean_squared_er
ror")
# Valor encontrado por validação cruzada
regressao_ridge_otimizada.fit(X_train, y_train)
regressao_ridge_otimizada.alpha_
np.sqrt(np.mean(-1 * score_ridge_otimizado))
```

Código-fonte 11 – *Tunning* regressão Ridge
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# otimizar alpha
regressao_lasso_otimizada = LassoCV(alphas = [0.001, 0.01, 0.1, 0.5, 0
.7, 1, 7, 10], cv=crossKfold)
score_lasso_otimizado = cross_val_score(regressao_lasso_otimizada, X_t
rain, y_train.values.ravel(),
                                        cv = crossKfold,
                                        scoring = "neg_mean_squared_er
ror")
# Valor encontrado por validação cruzada
regressao_lasso_otimizada.fit(X_train, y_train.values.ravel())
regressao_lasso_otimizada.alpha_
np.sqrt(np.mean(-1 * score_lasso_otimizado))
```

Código-fonte 12 – *Tunning* regressão LASSO
Fonte: Elaborado pelo autor (2022)

Após os algoritmos serem aplicados, foi possível ajustar 4 modelos:

1) Regressão Ridge.

2) Regressão Lasso.

3) Regressão Ridge Otimizada.

4) Regressão Lasso Otimizada.

Como etapa final, precisamos avaliar esses modelos na amostra de teste que separamos no início do processo. No Código-fonte “Obtenção dos resultados finais dos modelos”, resumimos as capacidades preditivas de cada algoritmo estimado. E como métrica de avaliação, utilizamos a Raiz do Erro Médio Quadrático.

```
# gerar previsoes
pred_ridge = regressao_ridge.predict(X_test)
pred_lasso = regressao_lasso.predict(X_test)
pred_ridge_otimizada = regressao_ridge_otimizada.predict(X_test)
pred_lasso_otimizada = regressao_lasso_otimizada.predict(X_test)

# obter o rmse
rmse_ridge = mean_squared_error(y_test, pred_ridge, squared=False)
rmse_lasso = mean_squared_error(y_test, pred_lasso, squared=False)
rmse_ridge_otimizada = mean_squared_error(y_test, pred_ridge_otimizada
, squared=False)
rmse_lasso_otimizada = mean_squared_error(y_test, pred_lasso_otimizada
, squared=False)

# apresentar os resultados
print(f'- RMSE Ridge: {rmse_ridge}')
print(f'- RMSE Lasso: {rmse_lasso}')
print(f'- RMSE Ridge otimizada: {rmse_ridge_otimizada}')
print(f'- RMSE Lasso otimizada: {rmse_lasso_otimizada}')
```

Código-fonte 13 – Obtenção dos resultados finais dos modelos
Fonte: Elaborado pelo autor (2022)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na Figura “Resultados das avaliações dos modelos”, apresentamos os resultados dos modelos treinados. Devido ao processo aleatório gerado na separação das amostras de treino e teste, podem existir diferenças nos valores da figura, e quando rodados novamente durante o estudo do código.

Em nosso exemplo, após otimizar os valores de alpha, foi possível perceber que o modelo de regressão Ridge otimizada demonstrou os menores valores de Raiz do Erro Médio Quadrático. Em vista disso, o uso da validação cruzada combinada com a regressão penalizada trouxe melhor capacidade preditiva para os algoritmos supervisionados.

```
- RMSE Ridge: 0.7425644242703652
- RMSE Lasso: 0.8282172542094777
- RMSE Ridge otimizada: 0.7294197361986285
- RMSE Lasso otimizada: 0.7447580207001858
```

Figura 9 – Resultados das avaliações dos modelos
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ATIVIDADE

Essa atividade não é avaliativa, caso queira ter um feedback, entre em contato com seu tutor pelo Teams.

O arquivo “psa_cancer_data.csv” foi disponibilizado, vamos utilizar as ferramentas para treinar nossos modelos usando validação cruzada e regressão penalizada.

Ao final, deve ser entregue um arquivo “.ipynb” contendo os códigos e respostas.

1. Acessar o *Colab*, carregar os módulos e os dados necessários.

2. Realizar as seguintes análises descritivas:

   a) Média, mediana e histograma de lpsa, lcavol e lweight.

   b) Correlação entre lpsa e as demais variáveis. Quais podem ser consideradas as mais relevantes?

3. Treinar um modelo LASSO com validação cruzada. Qual o melhor valor para alpha?

4. Treinar um modelo Ridge com validação cruzada. Qual o melhor valor para alpha?

5. Qual dos modelos apresentaram menor erro na amostra de teste?

Comentário: sugestão de respostas no arquivo “.ipynb” disponibilizado.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

GENÇAY, R.; SELÇUK, F.; WHITCHER, B. J. An introduction to wavelets and other filtering methods in finance and economics. Elsevier, 2001.

GEISSER, S. The predictive sample reuse method with applications. **Journal of the American Statistical Association**, v. 70, n. 350, pp. 320-328, 1975.

HASTIE, T.; TIBSHIRANI, R; FRIEDMAN, J. **The elements of statistical learning: data mining, inference and prediction**. 2nd. ed. New York: Springer, 2009.

HOERL, A.; KENNARD, R. Ridge regression. **Encyclopedia of Statistical Sciences.** v. 8, pp. 129–136, New York: Wiley, 1988.

JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. **An introduction to statistical learning**. New York: Springer, 2013.

STONE, M. Cross-validatory choice and assessment of statistical predictions. **Journal of the royal statistical society**: Series B (Methodological). v. 36, n. 2, pp. 111-133, 1974.

TIBSHIRANI, R. Regression shrinkage and selection via the lasso. **Journal of the Royal Statistical Society: Series B (Methodological)**. v. 58, n. 1, pp. 267-288, 1996.

ZOU, H.; HASTIE, T. Regularization and variable selection via the elastic net. **Journal of the royal statistical society**: series B (statistical methodology). v. 67, n. 2, pp. 301-320, 2005.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com